LLM-Text Watermarking based on Lagrange Interpolation



Manoj Kumar V K

Under the Guidance of

Dr. Shibam Ghosh

Department of Computer Science
Indian Insitute of Technogology Hyderabad

1. Introduction
2. Methodology
3. Extraction Variants
4. Security Analysis
5. Evaluation
6. Extensions
7. Conclusion
8. References

Introduction

- LLM-generated text can be modified, paraphrased, truncated, or partially deleted.
- Such modifications can disrupt watermark detection and authorship attribution.
- Watermarking embeds identifiable information into generated text without requiring a visible signature.
- **Problem:** Recover the embedded author identity even when only part of the original text survives.

Research Gaps

- Recovering a specific author's identity is more demanding than simply detecting a watermark.
- Text modifications can corrupt or remove embedded watermark information.
- The verifier must distinguish genuine watermark points from corrupted or spurious points.
- Reliable recovery requires a balance between robustness, computational efficiency, and security.

Contributions

- A Lagrange interpolation-based scheme that encodes an author's identity in a straight line over a finite field.
- A Maximum Collinear Points (MCP) approach to recover the watermark from potentially corrupted points.
- A security analysis of accidental collinearities and algebraic attacks.
- Extraction variants and extensions to multiple secrets and higher-degree polynomials.
- Experimental evaluation of random point sets over finite fields of different sizes.

Proposed Watermarking Scheme

1. Encode the secret identity as coefficients of a polynomial over $GF(2^n)$.
2. Generate pseudorandom coordinates x_i using a secret-seeded generator.
3. Compute watermark points $(x_i, f(x_i))$.
4. Embed the bits of these points into the generated text by biasing token-selection logits.
5. Recover candidate points from the text and use MCP to identify the dominant line.
6. Reconstruct the secret identity from the line's coefficients.

Watermark Representation

- All polynomial operations are performed over the finite field $GF(2^n)$.
- The secret identity is represented by the coefficients of a linear polynomial:

$$f(x) = a_0 + a_1 x$$

$$K = (a_0 \parallel a_1)$$

- Generate pseudorandom coordinates using an LFSR with secret seed sk:

$$x_i = \mathrm{LFSR}(t_i, sk)$$

- Compute the corresponding watermark points:

$$P_i = (x_i, y_i), \qquad y_i = f(x_i)$$

- Each point encodes $2n$ bits; N points require

$$T = 2nN \text{ bits.}$$

Embedding Watermark Bits into Text

- Partition the vocabulary into two subsets:

$$V = V_0 \cup V_1, \qquad V_0 \cap V_1 = \varnothing$$

- Tokens in V_0 represent bit 0, while tokens in V_1 represent bit 1.
- Modify token-selection logits to favor tokens corresponding to the desired watermark bit.
- The embedding procedure encodes both x_i and $y_i = f(x_i)$ into successive token blocks.
- The text remains naturally generated, while token choices carry the watermark information.

Algorithm 1 Watermark Embedding

Input: large language model LLM, vocabulary $\mathcal{V}$, $2n$-bit watermark $W = (a_0||a_1)$, bias δ, prompt $s^{(-N_p)}, \ldots, s^{(-1)}$, secret key $\mathcal{K}$, hash function H
Output: generated tokens $\mathcal{T}$ with embedded watermark W

$f(x) = a_1 x + a_0$ ▷ specify a polynomial based on the watermark
$\mathcal{T} = s^0 = LLM(s^{(-N_p)}, \ldots, s^{(-1)})$ ▷ generate the first token
$t = 1$ ▷ token counter
while (end of text generation) **do**
    $x = (x_0, x_1, \ldots, x_{n-1}) = H(s^{t-1}, \mathcal{K})$ ▷ generate pseudo-random x-coordinate
    $y = f(x) = (y_0, y_1, \ldots, y_{n-1})$ ▷ compute y-coordinate
    for $j = 0$ to n **do** ▷ a loop embedding y-coordinate into n consecutive tokens
        $seed = H(s^{t-1+j}, \mathcal{K})$
        Using a $seed$, pseudo-randomly split vocabulary $\mathcal{V}$ into two sets $\mathcal{V}_0$, $\mathcal{V}_1$
        Apply LLM to prior tokens $s^{(-N_p)} \ldots s^{(t-1+j)}$ to get a logit vector v over the vocabulary $\mathcal{V}$:
        $v = LLM(s^{(-N_p)}, \ldots, s^{(t-1+j)})$
        if $y_j == 0$ **then** $\hat{v}[w] = v[w] + \delta$ for all $w \in \mathcal{V}_0$ ▷ conditionally bias the logits
        else $\hat{v}[w] = v[w] + \delta$ for all $w \in \mathcal{V}_1$
        end if
        Sample the token $s^{(t+j)}$ from biased logits $\hat{v}$ and append it to $\mathcal{T}$
    end for
    $t = t + n$
end while
return $\mathcal{T}$

Algorithm 2 Watermark Extraction

Input: sequence of tokens $s^0, s^1, \ldots, s^N$, size of the watermark (in bits) $2n$, vocabulary $\mathcal{V}$, secret key $\mathcal{K}$, hash function H
Output: watermark $W = (a_0 \| a_1)$

$\mathbb{P} = \varnothing$ ▷ Create an empty list of points
$t = 1$ ▷ token counter
while (end of tokens) **do**
    $x = (x_0, x_1, \ldots, x_{n-1}) = H(s^{t-1}, \mathcal{K})$ ▷ extract x-coordinate
    for $j = 0$ to n **do** ▷ a loop extracting y-coordinate of a single point
        $seed = H(s^{t-1+j}, \mathcal{K})$
        Using a $seed$, pseudo-randomly split vocabulary $\mathcal{V}$ into two sets $\mathcal{V}_0, \mathcal{V}_1$
        if $s^t \in \mathcal{V}_0$ **then** $y_j = 0$ ▷ extract y_j bit of y-coordinate
        else $y_j = 1$
        end if
    end for
    $y = (y_0, y_1, \ldots, y_{n-1})$
    Append (x, y) point to the list $\mathbb{P}$
    $t = t + n$
end while
Find the maximum collinear points in $\mathbb{P}$
Apply the Lagrange interpolation to maximum collinear points to obtain $f(x) = a_0 + a_1 x$
return $W = (a_0 \| a_1)$

Maximum Collinear Points (MCP)

- Text modification may produce both genuine and corrupted watermark points.
- Genuine points lie on the original line:

$$f(x) = a_0 + a_1 x$$

- The MCP problem finds the largest subset of points lying on a common straight line.
- The line with the largest number of recovered points is considered the watermark candidate.

Simplified Watermark Extraction

- **Standard extraction**
  - Use the secret-keyed LFSR to identify valid x_i coordinates.
  - Extract the corresponding $f(x_i)$ values.
  - Apply MCP to the recovered points.
- **Simplified extraction**
  - Slide a $2n$-bit window across the extracted watermark bit sequence.
  - Interpret each window as a candidate point.
  - Apply MCP directly to all candidate points.
- This avoids filtering candidates using valid LFSR-generated coordinates but introduces more spurious points.

Double-Resolution Watermarking

- Consecutive points can be constructed using repeated applications of the linear function:

$$f(x) = a_0 + a_1 x$$

$$(x, f(x)),\ (f(x), f^2(x)),\ \ldots,\ (f^{N-1}(x), f^N(x))$$

- Consecutive points share coordinates, reducing the number of candidate points produced by sliding windows.
- Candidate count decreases from $2n(N-1)+1$ to $n(N-1)+1$.
- Short cycles may require a new initial value; the security implications must also be considered.

Security Analysis: Watermark Recovery

- Let $N = F + R$, where:
  - F: genuine points lying on the secret line.
  - R: random or corrupted points.
- The analysis models corrupted points as randomly distributed over $GF(2^n) \times GF(2^n)$.
- Recovery is reliable when the genuine line contains more points than any competing line formed by corrupted points.
- Larger finite fields reduce the probability of accidental collinearities under this model.

$$f(x) = a_0 + a_1 x, \qquad K = (a_0 \parallel a_1)$$

Algebraic Attack and Countermeasures

- **Vulnerability:** An LFSR and fixed token partitions can expose linear relationships in the watermark construction.
- **Potential attack:** An adversary may construct linear equations from observed token occurrences to infer the LFSR seed and watermark bits.
- **Countermeasures**
  - Replace the LFSR with a nonlinear feedback shift register (NFSR).
  - Use dynamic token partitions rather than fixed V_0 and V_1.
  - Derive partitions using cryptographic hashes and the secret key.
- Stronger security may introduce additional computational overhead.

Experimental Setup

- **Objective:** Evaluate the maximum number of accidental collinear points in random point sets.
- **Method:** Generate random points over finite fields of different sizes.
- **Fields:**
$$GF(2^6),\ GF(2^8),\ GF(2^{12}),\ GF(2^{16}),\ GF(2^{32})$$
- **Evaluation:** Measure the maximum number of collinear points for varying numbers of random points.
- **Goal:** Study how field size affects the likelihood of accidental watermark-like structures.

Experimental Results

Random Points	$GF(2^6)$	$GF(2^8)$	$GF(2^{12})$	$GF(2^{16})$	$GF(2^{32})$
8	3	2	2	2	2
16	3	3	2	2	2
32	5	3	2	2	2
64	6	4	3	2	2
128	6	5	3	2	2
256	10	5	3	2	2
512	14	8	3	2	2
1024	20	11	4	2	2

- Small fields exhibit more accidental collinearities.
- For $GF(2^{16})$ and $GF(2^{32})$, the observed maximum remains two in these experiments.
- Larger fields make accidental watermark-like structures less likely in the tested random sets.

Multiple Secrets and Higher-Degree Polynomials

- **Multiple lines as secrets**
  - Embed multiple lines, each encoding a distinct secret.
  - Recover the top t lines using MCP.
  - Establish a deterministic ordering before concatenating the secrets.
- **Higher-degree polynomials**

$$f(x) = a_0 + a_1 x + \cdots + a_d x^d$$

  - Encode the secret in the polynomial coefficients.
  - Reconstruct the polynomial using Lagrange interpolation.
  - Identify genuine points using Maximum Co-Polynomial Points (MCPP).

Conclusion and Future Work

- **Conclusion:**
  - Proposes Lagrange interpolation-based watermarking to encode author identity in LLM-generated text.
  - Uses MCP to recover the watermark from potentially modified text.
  - Improves security using cryptographically stronger pseudorandom generation and dynamic token partitions.
  - Supports extensions to multiple secrets and higher-degree polynomials.
- **Future work:**
  - Improve resilience against adversarial text modifications.
  - Optimize recovery of multiple secrets.
  - Explore hybrid schemes combining lines and higher-degree polynomials.

References


Jarosław Janas, Paweł Morawiecki, and Josef Pieprzyk.
Llm-text watermarking based on lagrange interpolation, 2026.

Thank You